Taller N°1

Tamara Benavidez

Tabla de Contenidos

ESCUELA POLITÉCNICA NACIONAL **1**

TALLER N°1 **2**

Ejercicio 1 2
Ejercicio 2 (Bubble sort) 4
Ejercicio 2.1 4
Ejercicio 2.2 6
Ejercicio 2.3 7
Ejercicio 3 10
Graficar 11
Ejercicio 4 14
Ejercicio 5 15
Ejercicio 6 17
Link del repositorio GitHub 18

ESCUELA POLITÉCNICA NACIONAL

TALLER N°1

Ejercicio 1

Algoritmo 01

Implemente en python

Ilustración

ENTRADA $N, x_1, x_2, \ldots, x_n$.

SALIDA

Paso 1 Tome $SUM = 0$. (*Inicialice el acumulador.*)

Paso 2 Para $i = 1, 2, \ldots, N$ hacer
Tome $SUM = SUM + x_i$. (*Añadir el siguiente término.*)

Paso 3 SALIDA (SUM);
PARE.

La sumatoria $1 + 1/2 + 1/4 + 1/8 \ldots$ tal que el error absoluto $e_{abs} < 10^{-1}$.

```python
# Valor real conocido
v_real = 2.0

# Umbral del error absoluto deseado
error_deseado = 1e-1

# Inicialización
SUM = 0.0
term = 1.0   # Primer término de la serie
i = 0

print("Iteración |   término   |    SUM     |   e_abs")
print("-------------------------------------------------")

# Genera términos hasta alcanzar la precisión deseada
while True:
    i += 1
    SUM += term
    e_abs = abs(v_real - SUM)
```

```python
    # Muestra los valores intermedios
    print(f"{i:9d} | {term:10.4f} | {SUM:9.4f} | {e_abs:7.4f}")

    if e_abs < error_deseado:
        break

    # El siguiente término es la mitad del actual (más eficiente que 1/(2**i))
    term *= 0.5
```

```
Iteración |   término   |    SUM     |   e_abs
-----------------------------------------------
        1 |     1.0000 |    1.0000 |  1.0000
        2 |     0.5000 |    1.5000 |  0.5000
        3 |     0.2500 |    1.7500 |  0.2500
        4 |     0.1250 |    1.8750 |  0.1250
        5 |     0.0625 |    1.9375 |  0.0625
```

```python
# Resultados finales
print("\n========= RESULTADOS =========")
print(f"Número de términos (N): {i}")
print(f"Suma aproximada (SUM): {SUM:.4f}")
print(f"Error absoluto (e_abs): {e_abs:.4f}")
print("==============================")
```

```
========= RESULTADOS =========
Número de términos (N): 5
Suma aproximada (SUM): 1.9375
Error absoluto (e_abs): 0.0625
==============================
```

Ejercicio 2 (Bubble sort)

Algoritmo 02

Implemente en python

- Pseudocode:

```
ARRAY a[0..n]
FOR (i = 0..n)
    swapped = false
    FOR (j = 1..n-i)
        IF (a[j] < a[j-1])
            swap(a, j, j-1)
            swapped = true
        END-IF
    END-FOR
    IF (! swapped)
        break
    END-IF
END-FOR
```

Ejercicio 2.1

$v_1 = [3, 2, 5, 8, 4, 1]$

i	Vector
0	$ [3, 2, 5, 8, 4, 1] $
1	$ [3, 2, 5, 4, 1, 8] $
2	$ [3, 2, 4, 1, 5, 8] $
3	$ [3, 2, 1, 4, 5, 8] $
...	...

Resultado final:

$v_{sorted} = [1, 2, 3, 4, 5, 8]$

```python
v1 = [3, 2, 5, 8, 4, 1]
n = len(v1)

print("VECTOR ORIGINAL:", v1)

for i in range(n):
    swapped = False
    print(f"\nIteración {i+1}:")
```

```python
    for j in range(1, n - i):
        print(f"  Comparando {v1[j-1]} y {v1[j]}", end=" → ")
        if v1[j] < v1[j - 1]:
            v1[j], v1[j - 1] = v1[j - 1], v1[j]
            swapped = True
            print("Intercambio →", v1)
        else:
            print("Sin cambio")
    if not swapped:
        print("  No hubo intercambios, el arreglo ya está ordenado.")
        break

print("\nVECTOR ORDENADO:", v1)
```

```
VECTOR ORIGINAL: [3, 2, 5, 8, 4, 1]

Iteración 1:
  Comparando 3 y 2 → Intercambio → [2, 3, 5, 8, 4, 1]
  Comparando 3 y 5 → Sin cambio
  Comparando 5 y 8 → Sin cambio
  Comparando 8 y 4 → Intercambio → [2, 3, 5, 4, 8, 1]
  Comparando 8 y 1 → Intercambio → [2, 3, 5, 4, 1, 8]

Iteración 2:
  Comparando 2 y 3 → Sin cambio
  Comparando 3 y 5 → Sin cambio
  Comparando 5 y 4 → Intercambio → [2, 3, 4, 5, 1, 8]
  Comparando 5 y 1 → Intercambio → [2, 3, 4, 1, 5, 8]

Iteración 3:
  Comparando 2 y 3 → Sin cambio
  Comparando 3 y 4 → Sin cambio
  Comparando 4 y 1 → Intercambio → [2, 3, 1, 4, 5, 8]

Iteración 4:
  Comparando 2 y 3 → Sin cambio
  Comparando 3 y 1 → Intercambio → [2, 1, 3, 4, 5, 8]

Iteración 5:
  Comparando 2 y 1 → Intercambio → [1, 2, 3, 4, 5, 8]

Iteración 6:
```

```
  No hubo intercambios, el arreglo ya está ordenado.

VECTOR ORDENADO: [1, 2, 3, 4, 5, 8]
```

Ejercicio 2.2

$v_2 = [-1, 0, 4, 5, 6, 7]$

```python
v2 = [-1, 0, 4, 5, 6, 7]
n = len(v2)

print("VECTOR ORIGINAL:", v2)

for i in range(n):
    swapped = False
    print(f"\nIteración {i+1}:")
    for j in range(1, n - i):
        print(f"  Comparando {v2[j-1]} y {v2[j]}", end=" → ")
        if v2[j] < v2[j - 1]:
            v2[j], v2[j - 1] = v2[j - 1], v2[j]
            swapped = True
            print("Intercambio →", v2)
        else:
            print("Sin cambio")
    if not swapped:
        print("  No hubo intercambios, el arreglo ya está ordenado.")
        break

print("\nVECTOR ORDENADO:", v2)
```

```
VECTOR ORIGINAL: [-1, 0, 4, 5, 6, 7]

Iteración 1:
  Comparando -1 y 0 → Sin cambio
  Comparando 0 y 4 → Sin cambio
  Comparando 4 y 5 → Sin cambio
  Comparando 5 y 6 → Sin cambio
  Comparando 6 y 7 → Sin cambio
  No hubo intercambios, el arreglo ya está ordenado.

VECTOR ORDENADO: [-1, 0, 4, 5, 6, 7]
```

Ejercicio 2.3

v_3 100_000 número aleatorios entre -200 y 145

```python
import random
import time

# Generar 100,000 números aleatorios entre -200 y 145
v3 = [random.randint(-200, 145) for _ in range(100_000)]
n = len(v3)

print("VECTOR ORIGINAL (primeros 10 elementos):", v3[:10])

inicio = time.time()
for i in range(n):
    swapped = False
    # Solo mostrar las primeras 10 iteraciones completas
    if i < 10:
        print(f"\nIteración {i+1}:")
    for j in range(1, n - i):
        if v3[j] < v3[j - 1]:
            v3[j], v3[j - 1] = v3[j - 1], v3[j]
            swapped = True
        # Mostrar solo comparaciones de las primeras 10 iteraciones y los primeros 10 elemen
        if i < 10 and j < 10:
            print(f"  Comparando índices {j-1} y {j}: {v3[j-1]} <-> {v3[j]}")
    if not swapped:
        if i < 10:
            print("  No hubo intercambios, el arreglo ya está ordenado.")
        break
fin = time.time()

print(f"\nTiempo total de ejecución: {fin - inicio:.2f} segundos")
print("Primeros 10 elementos ordenados:", v3[:10])
print("Últimos 10 elementos ordenados:", v3[-10:])
```

```
VECTOR ORIGINAL (primeros 10 elementos): [76, 83, -34, -35, 129, -190, 56, 8, -162, 67]

Iteración 1:
  Comparando índices 0 y 1: 76 <-> 83
  Comparando índices 1 y 2: -34 <-> 83
  Comparando índices 2 y 3: -35 <-> 83
```

```
  Comparando índices 3 y 4: 83 <-> 129
  Comparando índices 4 y 5: -190 <-> 129
  Comparando índices 5 y 6: 56 <-> 129
  Comparando índices 6 y 7: 8 <-> 129
  Comparando índices 7 y 8: -162 <-> 129
  Comparando índices 8 y 9: 67 <-> 129

Iteración 2:
  Comparando índices 0 y 1: -34 <-> 76
  Comparando índices 1 y 2: -35 <-> 76
  Comparando índices 2 y 3: 76 <-> 83
  Comparando índices 3 y 4: -190 <-> 83
  Comparando índices 4 y 5: 56 <-> 83
  Comparando índices 5 y 6: 8 <-> 83
  Comparando índices 6 y 7: -162 <-> 83
  Comparando índices 7 y 8: 67 <-> 83
  Comparando índices 8 y 9: 83 <-> 108

Iteración 3:
  Comparando índices 0 y 1: -35 <-> -34
  Comparando índices 1 y 2: -34 <-> 76
  Comparando índices 2 y 3: -190 <-> 76
  Comparando índices 3 y 4: 56 <-> 76
  Comparando índices 4 y 5: 8 <-> 76
  Comparando índices 5 y 6: -162 <-> 76
  Comparando índices 6 y 7: 67 <-> 76
  Comparando índices 7 y 8: 76 <-> 83
  Comparando índices 8 y 9: -199 <-> 83

Iteración 4:
  Comparando índices 0 y 1: -35 <-> -34
  Comparando índices 1 y 2: -190 <-> -34
  Comparando índices 2 y 3: -34 <-> 56
  Comparando índices 3 y 4: 8 <-> 56
  Comparando índices 4 y 5: -162 <-> 56
  Comparando índices 5 y 6: 56 <-> 67
  Comparando índices 6 y 7: 67 <-> 76
  Comparando índices 7 y 8: -199 <-> 76
  Comparando índices 8 y 9: -111 <-> 76

Iteración 5:
  Comparando índices 0 y 1: -190 <-> -35
  Comparando índices 1 y 2: -35 <-> -34
```

```
  Comparando índices 2 y 3: -34 <-> 8
  Comparando índices 3 y 4: -162 <-> 8
  Comparando índices 4 y 5: 8 <-> 56
  Comparando índices 5 y 6: 56 <-> 67
  Comparando índices 6 y 7: -199 <-> 67
  Comparando índices 7 y 8: -111 <-> 67
  Comparando índices 8 y 9: -143 <-> 67

Iteración 6:
  Comparando índices 0 y 1: -190 <-> -35
  Comparando índices 1 y 2: -35 <-> -34
  Comparando índices 2 y 3: -162 <-> -34
  Comparando índices 3 y 4: -34 <-> 8
  Comparando índices 4 y 5: 8 <-> 56
  Comparando índices 5 y 6: -199 <-> 56
  Comparando índices 6 y 7: -111 <-> 56
  Comparando índices 7 y 8: -143 <-> 56
  Comparando índices 8 y 9: -172 <-> 56

Iteración 7:
  Comparando índices 0 y 1: -190 <-> -35
  Comparando índices 1 y 2: -162 <-> -35
  Comparando índices 2 y 3: -35 <-> -34
  Comparando índices 3 y 4: -34 <-> 8
  Comparando índices 4 y 5: -199 <-> 8
  Comparando índices 5 y 6: -111 <-> 8
  Comparando índices 6 y 7: -143 <-> 8
  Comparando índices 7 y 8: -172 <-> 8
  Comparando índices 8 y 9: -59 <-> 8

Iteración 8:
  Comparando índices 0 y 1: -190 <-> -162
  Comparando índices 1 y 2: -162 <-> -35
  Comparando índices 2 y 3: -35 <-> -34
  Comparando índices 3 y 4: -199 <-> -34
  Comparando índices 4 y 5: -111 <-> -34
  Comparando índices 5 y 6: -143 <-> -34
  Comparando índices 6 y 7: -172 <-> -34
  Comparando índices 7 y 8: -59 <-> -34
  Comparando índices 8 y 9: -34 <-> 8

Iteración 9:
  Comparando índices 0 y 1: -190 <-> -162
```

```
  Comparando índices 1 y 2: -162 <-> -35
  Comparando índices 2 y 3: -199 <-> -35
  Comparando índices 3 y 4: -111 <-> -35
  Comparando índices 4 y 5: -143 <-> -35
  Comparando índices 5 y 6: -172 <-> -35
  Comparando índices 6 y 7: -59 <-> -35
  Comparando índices 7 y 8: -35 <-> -34
  Comparando índices 8 y 9: -34 <-> 8

Iteración 10:
  Comparando índices 0 y 1: -190 <-> -162
  Comparando índices 1 y 2: -199 <-> -162
  Comparando índices 2 y 3: -162 <-> -111
  Comparando índices 3 y 4: -143 <-> -111
  Comparando índices 4 y 5: -172 <-> -111
  Comparando índices 5 y 6: -111 <-> -59
  Comparando índices 6 y 7: -59 <-> -35
  Comparando índices 7 y 8: -35 <-> -34
  Comparando índices 8 y 9: -34 <-> 0
```

Ejercicio 3

Algoritmo 03 | Participación en clase

Resuelva

Pseudocode:

```
procedure iterative          (n: nonnegative integer)
if n=0 then
    return 0
else
   x := 0
   y := 1
   for i := 1 to n − 1
        z := x + y
        x := y
        y := z
return y
```

$$n = 11, y = ?$$

n	fib(n)
0	0
1	1

n	fib(n)
2	1
3	2
4	3
5	5
6	8
7	13
...	...
n = 11	?
n = 84	?
n = 1531	?

```python
def fibonacci_iterativo(n):
    if n == 0:
        return 0

    x = 0
    y = 1

    for i in range(1, n):
        z = x + y
        x = y
        y = z

    return y

# Resultados
for n in [11, 84, 1531]:
    print(f"fib({n}) = {fibonacci_iterativo(n)}")
```

```
fib(11) = 89
fib(84) = 160500643816367088
fib(1531) = 407936176052377669101778911015323059541693566794692519680122463207854422013990100
```

Graficar

- El valor de la serie $fib(n)$
- El valor del cociente

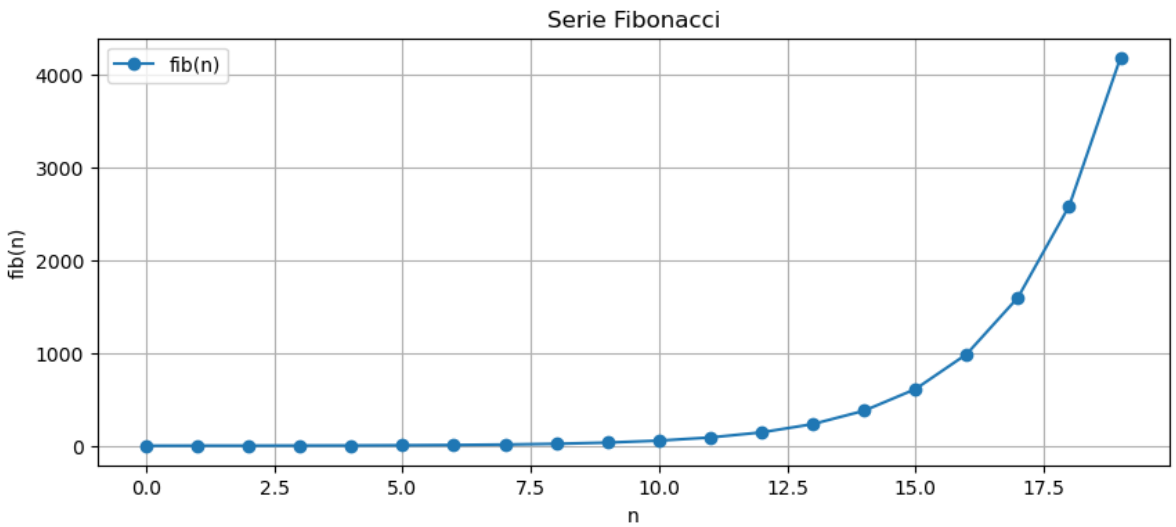
  $$\phi \;\rightarrow\; \frac{\text{fib}(n)}{\text{fib}(n-1)} \;\approx\; 1.618 \text{ (número áureo)}$$

n	$\frac{\text{fib}(n)}{\text{fib}(n-1)}$
2	$\frac{1}{1} = 1$
3	$\frac{2}{1} = 2$
4	$\frac{3}{2} = 1.5$
5	$\frac{5}{3} = 1.66667$
6	$\frac{8}{5} = 1.6$
7	$\frac{13}{8} = 1.625$
8	$\frac{21}{13} = 1.615$
...	...
∞	$\frac{1+\sqrt{5}}{2} \approx 1.618$ (número áureo)

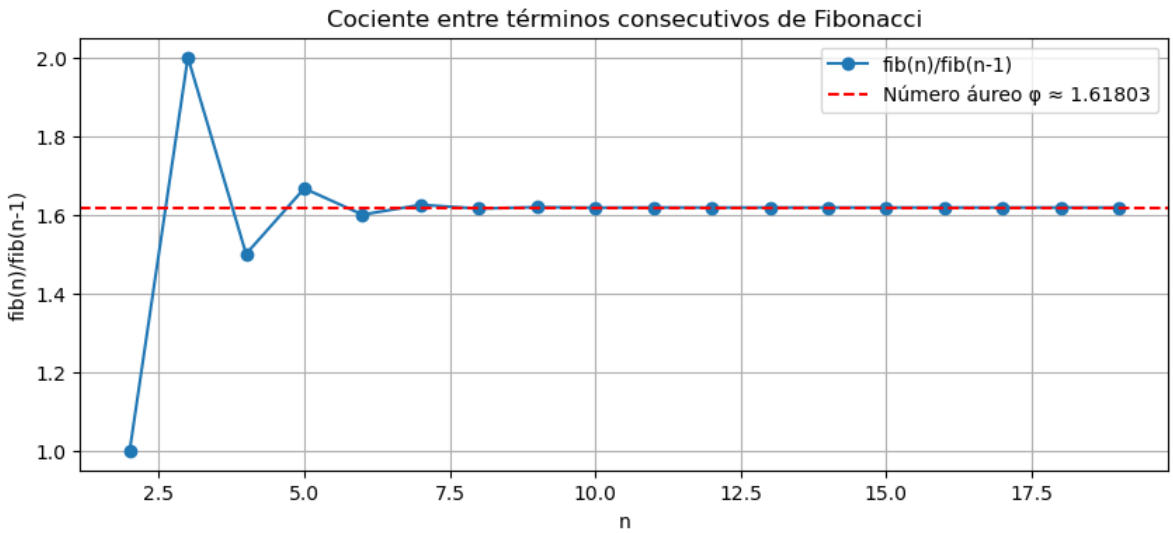
```python
import matplotlib.pyplot as plt

# Función para generar la serie Fibonacci hasta n términos
def fibonacci(n):
    fib = [0, 1]  # valores iniciales
    for i in range(2, n):
        fib.append(fib[-1] + fib[-2])
    return fib

# Número de términos
n = 20
fib_series = fibonacci(n)

# Calcular los cocientes fib(n)/fib(n-1)
ratios = [fib_series[i] / fib_series[i - 1] for i in range(2, n)]

# Valor teórico del número áureo
phi = (1 + 5 ** 0.5) / 2
```

```python
# 1 Gráfico de la serie Fibonacci
plt.figure(figsize=(10, 4))
plt.plot(range(n), fib_series, marker='o', label='fib(n)')
plt.title("Serie Fibonacci")
plt.xlabel("n")
```

```
plt.ylabel("fib(n)")
plt.grid(True)
plt.legend()
plt.show()
```




```
# 2 Gráfico del cociente fib(n)/fib(n-1)
plt.figure(figsize=(10, 4))
plt.plot(range(2, n), ratios, marker='o', label='fib(n)/fib(n-1)')
plt.axhline(y=phi, color='r', linestyle='--', label=f'Número áureo    {phi:.5f}')
plt.title("Cociente entre términos consecutivos de Fibonacci")
plt.xlabel("n")
plt.ylabel("fib(n)/fib(n-1)")
plt.grid(True)
plt.legend()
plt.show()
```



Ejercicio 4

Modifique el Algoritmo 02 y determine el número de comparaciones realizadas al ordenar la serie 5, 4, 3, 2, 1

```
v1 = [3, 2, 5, 4, 1]
n = len(v1)
comparaciones = 0  # Contador de comparaciones

print("VECTOR ORIGINAL:", v1)

for i in range(n):
    swapped = False
    print(f"\nIteración {i+1}:")
    for j in range(1, n - i):
        comparaciones += 1  # Se realiza una comparación
        print(f"  Comparando {v1[j-1]} y {v1[j]}", end=" → ")
        if v1[j] < v1[j - 1]:
            v1[j], v1[j - 1] = v1[j - 1], v1[j]
            swapped = True
            print("Intercambio →", v1)
        else:
            print("Sin cambio")
    if not swapped:
        print("  No hubo intercambios, el arreglo ya está ordenado.")
```

```
        break

print("\nVECTOR ORDENADO:", v1)
print("NÚMERO DE COMPARACIONES REALIZADAS:", comparaciones)
```

```
VECTOR ORIGINAL: [3, 2, 5, 4, 1]

Iteración 1:
  Comparando 3 y 2 → Intercambio → [2, 3, 5, 4, 1]
  Comparando 3 y 5 → Sin cambio
  Comparando 5 y 4 → Intercambio → [2, 3, 4, 5, 1]
  Comparando 5 y 1 → Intercambio → [2, 3, 4, 1, 5]

Iteración 2:
  Comparando 2 y 3 → Sin cambio
  Comparando 3 y 4 → Sin cambio
  Comparando 4 y 1 → Intercambio → [2, 3, 1, 4, 5]

Iteración 3:
  Comparando 2 y 3 → Sin cambio
  Comparando 3 y 1 → Intercambio → [2, 1, 3, 4, 5]

Iteración 4:
  Comparando 2 y 1 → Intercambio → [1, 2, 3, 4, 5]

Iteración 5:
  No hubo intercambios, el arreglo ya está ordenado.

VECTOR ORDENADO: [1, 2, 3, 4, 5]
NÚMERO DE COMPARACIONES REALIZADAS: 10
```

Ejercicio 5

Usando el Algoritmo 03 y la aritmética de redondeo con 3 cifras. Determine la iteración desde la cual el error relativo de

$$\frac{y_{i-1}}{y_i} \quad (i > 0) \text{ con respecto a } \frac{1+\sqrt{5}}{2} \text{ está dentro de } 10^{-5}$$

```
def FL_rounding(x, digits=3):
    """Redondeo a 'digits' cifras significativas para float"""
```

```python
    if x == 0:
        return 0.0
    else:
        import math
        return round(x, digits - int(math.floor(math.log10(abs(x)))) - 1)

def fibonacci_iterativo_FL_seguro(n, digits=3):
    """Fibonacci iterativo usando FL_rounding solo para la razón"""
    if n == 0:
        return 0, None, None
    x = 0  # enteros grandes
    y = 1
    phi = (1 + 5**0.5)/2
    iteracion_error = None
    ratio = None

    for i in range(1, n+1):
        z = x + y
        x = y
        y = z
        if i > 1:
            # convertimos a float para calcular la razón y redondear
            ratio_float = y / x
            ratio = FL_rounding(ratio_float, digits)
            error = abs(phi - ratio) / phi
            if iteracion_error is None and error < 1e-5:
                iteracion_error = i
    return y, ratio, iteracion_error

# Valores de n que queremos probar
valores_n = [11, 84, 1531]

print("   n   | fib(n) | ratio y_i/y_{i-1} | iteración error < 1e-5")
print("-------|----------------|-------------------|-----------------------")
for n in valores_n:
    fib_n, ratio, iter_error = fibonacci_iterativo_FL_seguro(n, digits=3)
    print(f"{n}   | {fib_n} | {ratio} | {iter_error}")
```

```
   n   | fib(n) | ratio y_i/y_{i-1} | iteración error < 1e-5
-------|----------------|-------------------|-----------------------
11   | 144 | 1.62 | None
84   | 259695496911122585 | 1.62 | None
```

1531 | 660054598093407973636853072375872673199693713421877595011852807782292761397911544407

Ejercicio 6

Implemente la serie geométrica

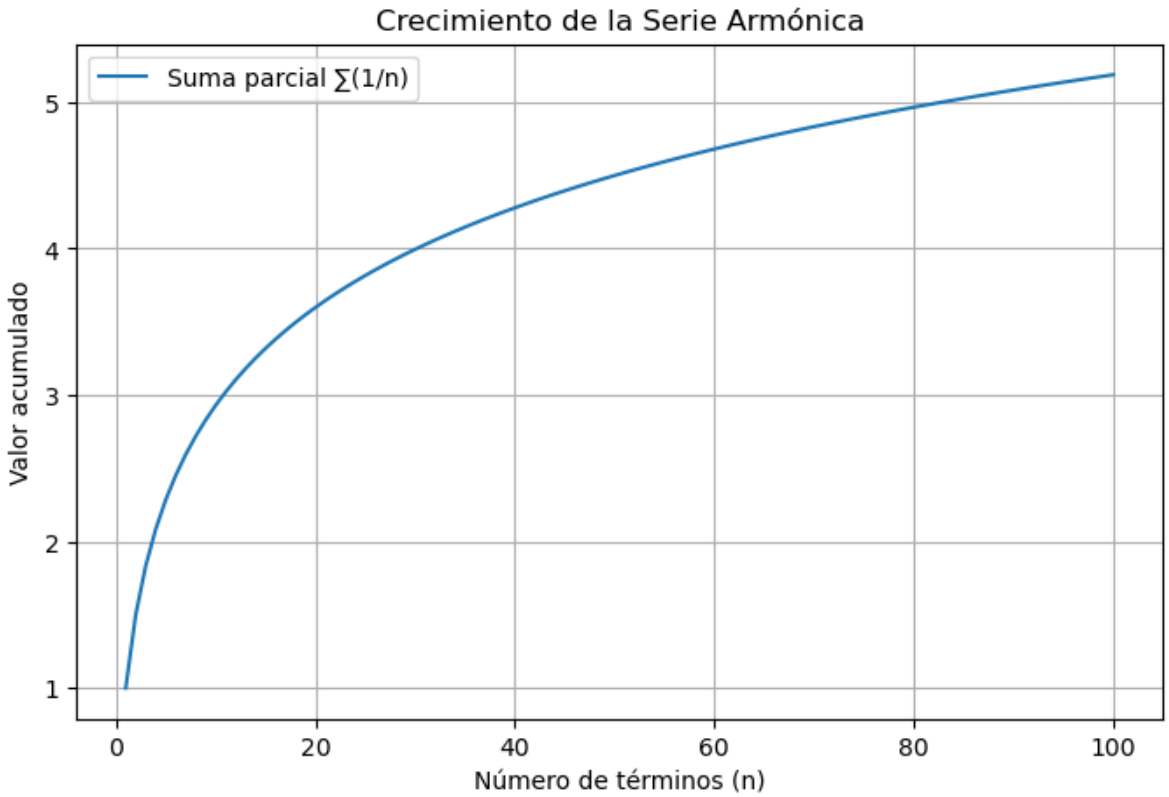
$\sum_{n=1}^{\infty} \frac{1}{n}$

A qué valor converge?

El Algoritmo 04 implementa la serie armónica, no converge: diverge y su suma tiende a $+\infty$

```
import matplotlib.pyplot as plt

# Número de términos
N = int(input("Ingrese el número de términos N: "))

suma = 0.0
valores = []  # Para guardar las sumas parciales

for n in range(1, N + 1):
    suma += 1 / n
    valores.append(suma)

# Mostrar el último valor
print(f"Suma parcial con N={N}: {suma:.6f}")

# Graficar la serie
plt.figure(figsize=(8, 5))
plt.plot(range(1, N + 1), valores, label='Suma parcial  (1/n)')
plt.title('Crecimiento de la Serie Armónica')
plt.xlabel('Número de términos (n)')
plt.ylabel('Valor acumulado')
plt.grid(True)
plt.legend()
plt.show()
```

```
Ingrese el número de términos N:  100

Suma parcial con N=100: 5.187378
```



Link del repositorio GitHub

github_TamyBenavidez, Taller N1